

Министерство науки и высшего образования Российской Федерации
ФГАОУ ВО «УрФУ имени первого Президента России Б.Н. Ельцина»
Институт радиоэлектроники и информационных технологий – РТФ

«Введение в функциональное программирование и Stream API»

Лабораторное задание №5

Группа: РИМ-150950

Студент: Владимиров Н.А.

Преподаватель: Архипов Н. А.

Екатеринбург

2026

1. Цель работы

Получить представление о функциональном программировании и Stream API в Java, научиться применять методы `filter()`, `map()`, `anyMatch()` и лямбда-выражения для обработки массивов и списков.

2. Описание задачи

В разделе 1 методички приведены примеры решения задач:

1. фильтрация чётных чисел из массива;
2. поиск общих элементов двух массивов;
3. отбор строк, начинающихся с заглавной буквы;
4. получение списка квадратов чисел.

В разделе 2 необходимо реализовать все примеры и дополнительные задания:

5. оставить строки, содержащие заданную подстроку;
6. оставить числа, делящиеся на заданное число без остатка;
7. оставить строки длиннее заданного значения;
8. оставить числа, большие заданного значения;
9. оставить строки, состоящие только из букв;
10. оставить числа, меньшие заданного значения;
11. решить две задачи с сайта acm.timus.ru.

3. Ход выполнения

Создан Java-проект lab_5. Примеры из раздела 1 размещены в пакете examples_lr5 (классы Example1–Example4). Для самостоятельной работы общие функции вынесены в класс StreamFunctions, демонстрация выполнена в классах Task1–Task10. Запуск всех примеров и заданий выполняется через класс Main.

Репозиторий с исходным кодом: https://github.com/n0vision/Java_Labs_2026

3.1. Примеры решения задач (раздел 1)

Пример 1 (класс examples_lr5.Example1):

```
package examples_lr5;

import java.util.Arrays;

public class Example1 {

    public static int[] filterEvenNumbers(int[] numbers) {
        return Arrays.stream(numbers)
            .filter(n -> n % 2 == 0)
            .toArray();
    }

    public static boolean hasEvenNumber(int[] numbers) {
        return Arrays.stream(numbers).anyMatch(n -> n % 2 == 0);
    }

    public static void main(String[] args) {
        int[] numbers = {1, 2, 3, 4, 5, 6, 7, 8};
        int[] evenNumbers = filterEvenNumbers(numbers);

        System.out.println("Пример 1. Четные числа из массива");
        System.out.println("Исходный массив: " + Arrays.toString(numbers));
        System.out.println("Результат filter(): " +
Arrays.toString(evenNumbers));
        System.out.println("Есть четные числа (anyMatch): " +
```

```
hasEvenNumber(numbers));  
    }  
}
```

Пример 2 (класс examples_lr5.Example2):

```
package examples_lr5;  
  
import java.util.Arrays;  
import java.util.Set;  
import java.util.stream.Collectors;  
  
public class Example2 {  
  
    public static int[] commonElements(int[] first, int[] second) {  
        Set<Integer> secondSet = Arrays.stream(second)  
            .boxed()  
            .collect(Collectors.toSet());  
  
        return Arrays.stream(first)  
            .filter(secondSet::contains)  
            .distinct()  
            .toArray();  
    }  
  
    public static void main(String[] args) {  
        int[] first = {1, 2, 3, 4, 5};  
        int[] second = {3, 4, 5, 6, 7};  
        int[] common = commonElements(first, second);  
  
        System.out.println("Пример 2. Общие элементы двух массивов");  
        System.out.println("Первый массив: " + Arrays.toString(first));  
        System.out.println("Второй массив: " + Arrays.toString(second));  
        System.out.println("Результат filter(): " + Arrays.toString(common));  
    }  
}
```

Пример 3 (класс examples_lr5.Example3):

```

package examples_lr5;

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class Example3 {

    public static List<String> filterUpperCaseStart(List<String> strings) {
        return strings.stream()
            .filter(s -> !s.isEmpty() && Character.isUpperCase(s.charAt(0)))
            .collect(Collectors.toList());
    }

    public static void main(String[] args) {
        List<String> strings = Arrays.asList("Java", "python", "Stream", "api",
"Lambda");
        List<String> result = filterUpperCaseStart(strings);

        System.out.println("Пример 3. Строки, начинающиеся с большой буквы");
        System.out.println("Исходный список: " + strings);
        System.out.println("Результат filter(): " + result);
    }
}

```

Пример 4 (класс examples_lr5.Example4):

```

package examples_lr5;

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class Example4 {

    public static List<Integer> mapToSquares(List<Integer> numbers) {
        return numbers.stream()
            .map(n -> n * n)
            .collect(Collectors.toList());
    }
}

```

```

    }

    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
        List<Integer> squares = mapToSquares(numbers);

        System.out.println("Пример 4. Квадраты чисел");
        System.out.println("Исходный список: " + numbers);
        System.out.println("Результат map(): " + squares);
    }
}

```

3.2. Класс StreamFunctions

```

package lr5;

import java.util.Arrays;
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;

public final class StreamFunctions {

    private StreamFunctions() {
    }

    public static int[] filterEvenNumbers(int[] numbers) {
        return Arrays.stream(numbers)
            .filter(n -> n % 2 == 0)
            .toArray();
    }

    public static int[] commonElements(int[] first, int[] second) {
        Set<Integer> secondSet = Arrays.stream(second)
            .boxed()
            .collect(Collectors.toSet());

        return Arrays.stream(first)
            .filter(secondSet::contains)

```

```

        .distinct()
        .toArray();
    }

    public static List<String> filterUpperCaseStart(List<String> strings) {
        return strings.stream()
            .filter(s -> !s.isEmpty() && Character.isUpperCase(s.charAt(0)))
            .collect(Collectors.toList());
    }

    public static List<Integer> mapToSquares(List<Integer> numbers) {
        return numbers.stream()
            .map(n -> n * n)
            .collect(Collectors.toList());
    }

    public static List<String> filterBySubstring(List<String> strings, String
substring) {
        return strings.stream()
            .filter(s -> s.contains(substring))
            .collect(Collectors.toList());
    }

    public static List<Integer> filterDivisibleBy(List<Integer> numbers, int
divisor) {
        return numbers.stream()
            .filter(n -> divisor != 0 && n % divisor == 0)
            .collect(Collectors.toList());
    }

    public static List<String> filterLongerThan(List<String> strings, int
minLength) {
        return strings.stream()
            .filter(s -> s.length() > minLength)
            .collect(Collectors.toList());
    }

    public static List<Integer> filterGreaterThan(List<Integer> numbers, int
threshold) {
        return numbers.stream()

```

```

        .filter(n -> n > threshold)
        .collect(Collectors.toList());
    }

    public static List<String> filterLettersOnly(List<String> strings) {
        return strings.stream()
            .filter(s -> !s.isEmpty() &&
s.chars().allMatch(Character::isLetter))
            .collect(Collectors.toList());
    }

    public static List<Integer> filterLessThan(List<Integer> numbers, int
threshold) {
        return numbers.stream()
            .filter(n -> n < threshold)
            .collect(Collectors.toList());
    }

    public static boolean hasEvenNumber(int[] numbers) {
        return Arrays.stream(numbers).anyMatch(n -> n % 2 == 0);
    }
}

```

3.3. Задания для самостоятельной работы

Задание 1 (класс `lr5.task1.Task1`):

```

package lr5.task1;

import lr5.StreamFunctions;

import java.util.Arrays;

public class Task1 {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3, 4, 5, 6, 7, 8};
        int[] evenNumbers = StreamFunctions.filterEvenNumbers(numbers);

        System.out.println("Исходный массив: " + Arrays.toString(numbers));
    }
}

```



```

        System.out.println("Четные числа: " + Arrays.toString(evenNumbers));
        System.out.println("Есть четные числа (anyMatch): " +
StreamFunctions.hasEvenNumber(numbers));
    }
}

```

Задание 2 (класс `lr5.task2.Task2`):

```

package lr5.task2;

import lr5.StreamFunctions;

import java.util.Arrays;

public class Task2 {
    public static void main(String[] args) {
        int[] first = {1, 2, 3, 4, 5};
        int[] second = {3, 4, 5, 6, 7};
        int[] common = StreamFunctions.commonElements(first, second);

        System.out.println("Первый массив: " + Arrays.toString(first));
        System.out.println("Второй массив: " + Arrays.toString(second));
        System.out.println("Общие элементы: " + Arrays.toString(common));
    }
}

```

Задание 3 (класс `lr5.task3.Task3`):

```

package lr5.task3;

import lr5.StreamFunctions;

import java.util.Arrays;
import java.util.List;

public class Task3 {
    public static void main(String[] args) {
        List<String> strings = Arrays.asList("Java", "python", "Stream", "api",

```

```

"Lambda");
        List<String> upperCaseStart =
StreamFunctions.filterUpperCaseStart(strings);

        System.out.println("Исходный список: " + strings);
        System.out.println("Строки с большой буквы: " + upperCaseStart);
    }
}

```

Задание 4 (класс lr5.task4.Task4):

```

package lr5.task4;

import lr5.StreamFunctions;

import java.util.Arrays;
import java.util.List;

public class Task4 {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, -6);
        List<Integer> squares = StreamFunctions.mapToSquares(numbers);

        System.out.println("Исходный список: " + numbers);
        System.out.println("Квадраты чисел: " + squares);
    }
}

```

Задание 5 (класс lr5.task5.Task5):

```

package lr5.task5;

import lr5.StreamFunctions;

import java.util.Arrays;
import java.util.List;

public class Task5 {

```

```

    public static void main(String[] args) {
        List<String> strings = Arrays.asList("Java Stream", "Python", "Stream
API", "Lambda");
        String substring = "Stream";
        List<String> result = StreamFunctions.filterBySubstring(strings,
substring);

        System.out.println("Исходный список: " + strings);
        System.out.println("Подстрока: " + substring);
        System.out.println("Строки с подстрокой: " + result);
    }
}

```

Задание 6 (класс lr5.task6.Task6):

```

package lr5.task6;

import lr5.StreamFunctions;

import java.util.Arrays;
import java.util.List;

public class Task6 {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(10, 15, 20, 25, 30, 33);
        int divisor = 5;
        List<Integer> result = StreamFunctions.filterDivisibleBy(numbers,
divisor);

        System.out.println("Исходный список: " + numbers);
        System.out.println("Делитель: " + divisor);
        System.out.println("Числа, делящиеся без остатка: " + result);
    }
}

```

Задание 7 (класс lr5.task7.Task7):

```

package lr5.task7;

import lr5.StreamFunctions;

import java.util.Arrays;
import java.util.List;

public class Task7 {
    public static void main(String[] args) {
        List<String> strings = Arrays.asList("a", "ab", "abc", "abcd", "hello");
        int minLength = 3;
        List<String> result = StreamFunctions.filterLongerThan(strings,
minLength);

        System.out.println("Исходный список: " + strings);
        System.out.println("Минимальная длина: " + minLength);
        System.out.println("Строки длинее заданного значения: " + result);
    }
}

```

Задание 8 (класс lr5.task8.Task8):

```

package lr5.task8;

import lr5.StreamFunctions;

import java.util.Arrays;
import java.util.List;

public class Task8 {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(-5, 0, 3, 7, 10, 15);
        int threshold = 5;
        List<Integer> result = StreamFunctions.filterGreaterThan(numbers,
threshold);

        System.out.println("Исходный список: " + numbers);
        System.out.println("Пороговое значение: " + threshold);
        System.out.println("Числа больше порога: " + result);
    }
}

```

```
    }  
}
```

Задание 9 (класс lr5.task9.Task9):

```
package lr5.task9;  
  
import lr5.StreamFunctions;  
  
import java.util.Arrays;  
import java.util.List;  
  
public class Task9 {  
    public static void main(String[] args) {  
        List<String> strings = Arrays.asList("Java", "Stream123", "API",  
"test!", "Lambda");  
        List<String> result = StreamFunctions.filterLettersOnly(strings);  
  
        System.out.println("Исходный список: " + strings);  
        System.out.println("Строки только из букв: " + result);  
    }  
}
```

Задание 10 (класс lr5.task10.Task10):

```
package lr5.task10;  
  
import lr5.StreamFunctions;  
  
import java.util.Arrays;  
import java.util.List;  
  
public class Task10 {  
    public static void main(String[] args) {  
        List<Integer> numbers = Arrays.asList(-5, 0, 3, 7, 10, 15);  
        int threshold = 5;  
        List<Integer> result = StreamFunctions.filterLessThan(numbers,  
threshold);  
    }  
}
```

```

        System.out.println("Исходный список: " + numbers);
        System.out.println("Пороговое значение: " + threshold);
        System.out.println("Числа меньше порога: " + result);
    }
}

```

3.4. Класс Main

```

package lr5;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== Лабораторная работа №5: Stream API ===\n");

        System.out.println("--- Примеры решения задач ---\n");
        examples_lr5.Example1.main(args);
        System.out.println();

        examples_lr5.Example2.main(args);
        System.out.println();

        examples_lr5.Example3.main(args);
        System.out.println();

        examples_lr5.Example4.main(args);
        System.out.println();

        System.out.println("--- Задания для самостоятельной работы ---\n");
        lr5.task1.Task1.main(args);
        System.out.println();

        lr5.task2.Task2.main(args);
        System.out.println();

        lr5.task3.Task3.main(args);
        System.out.println();

        lr5.task4.Task4.main(args);
    }
}

```

```

        System.out.println();

        lr5.task5.Task5.main(args);
        System.out.println();

        lr5.task6.Task6.main(args);
        System.out.println();

        lr5.task7.Task7.main(args);
        System.out.println();

        lr5.task8.Task8.main(args);
        System.out.println();

        lr5.task9.Task9.main(args);
        System.out.println();

        lr5.task10.Task10.main(args);
    }
}

```

3.5. Задачи Timus

Задача 1877 «HTTP Requests»: сравнение количества GET- и POST-запросов.

```

package timus.task_1877;

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int getRequests = scanner.nextInt();
        int postRequests = scanner.nextInt();

        if (getRequests > postRequests) {
            System.out.println("GET");
        } else if (getRequests < postRequests) {
            System.out.println("POST");
        }
    }
}

```

```

        } else {
            System.out.println("EQUAL");
        }

        scanner.close();
    }
}

```

Задача 1263 «Разминка»: подсчёт количества чётных чисел в последовательности.

```

package timus.task_1263;

import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int n = scanner.nextInt();
        int[] numbers = new int[n];

        for (int i = 0; i < n; i++) {
            numbers[i] = scanner.nextInt();
        }

        long evenCount = Arrays.stream(numbers)
            .filter(value -> value % 2 == 0)
            .count();

        System.out.println(evenCount);
        scanner.close();
    }
}

```

4. Вывод

В ходе выполнения лабораторной работы изучены основы функционального программирования в Java и Stream API. Реализованы примеры из раздела 1 и все задания для самостоятельной работы с использованием методов `filter()`, `map()`, `anyMatch()` и лямбда-выражений. Программа успешно компилируется и выполняется. Дополнительно решены задачи 1877 и 1263 с сайта [timus](https://timus.solver.ru/)

5. Результаты выполнения программы

--- Примеры решения задач ---

Пример 1. Четные числа из массива

Исходный массив: [1, 2, 3, 4, 5, 6, 7, 8]

Результат `filter()`: [2, 4, 6, 8]

Есть четные числа (`anyMatch()`): true

Пример 2. Общие элементы двух массивов

Первый массив: [1, 2, 3, 4, 5]

Второй массив: [3, 4, 5, 6, 7]

Результат `filter()`: [3, 4, 5]

Пример 3. Строки, начинающиеся с большой буквы

Исходный список: [Java, python, Stream, api, Lambda]

Результат `filter()`: [Java, Stream, Lambda]

Пример 4. Квадраты чисел

Исходный список: [1, 2, 3, 4, 5]

Результат `map()`: [1, 4, 9, 16, 25]

--- Задания для самостоятельной работы ---

Исходный массив: [1, 2, 3, 4, 5, 6, 7, 8]

Четные числа: [2, 4, 6, 8]

Есть четные числа (anyMatch): true

Первый массив: [1, 2, 3, 4, 5]

Второй массив: [3, 4, 5, 6, 7]

Общие элементы: [3, 4, 5]

Исходный список: [Java, python, Stream, api, Lambda]

Строки с большой буквы: [Java, Stream, Lambda]

Исходный список: [1, 2, 3, 4, 5, -6]

Квадраты чисел: [1, 4, 9, 16, 25, 36]

Исходный список: [Java Stream, Python, Stream API, Lambda]

Подстрока: Stream

Строки с подстрокой: [Java Stream, Stream API]

Исходный список: [10, 15, 20, 25, 30, 33]

Делитель: 5

Числа, делящиеся без остатка: [10, 15, 20, 25, 30]

Исходный список: [a, ab, abc, abcd, hello]

Минимальная длина: 3

Строки длиннее заданного значения: [abcd, hello]

Исходный список: [-5, 0, 3, 7, 10, 15]

Пороговое значение: 5

Числа больше порога: [7, 10, 15]

Исходный список: [Java, Stream123, API, test!, Lambda]

Строки только из букв: [Java, API, Lambda]

Исходный список: [-5, 0, 3, 7, 10, 15]

Пороговое значение: 5

Числа меньше порога: [-5, 0, 3]